Real-Time Embedded Systems

Final Assignment Report

AERO4 — 2025/2026

Eugénie CHEVALIER

April 2026

Professor: Jasdeep Singh

Table of Contents

1. Introduction 3
2. Task τ_1 — Multiplication & Measurement 3
   2.1 Implementation 3
   2.2 Measurement Results 3
   2.3 Determination of C_1 4
3. Task Set & Schedulability Analysis 4
   3.1 Task Set Definition 4
   3.2 Utilization Analysis 5
   3.3 Schedulability Proof 5
4. Non-Preemptive Schedule Search 5
   4.1 Algorithm Description 5
   4.2 Computational Complexity 5
   4.3 Part 1 — No Missed Deadlines 6
   4.4 Part 2 — τ_5 May Miss a Deadline 7
      4.4.1 Why Part 2 = Part 1 7
5. Job-Level Response Time Verification 8
6. Assumptions & Design Decisions 9
7. Conclusion 10

1. Introduction

The objective of this present report is to design and analyze a non-preemptive scheduling solution for a set of 7 periodic tasks with implicit deadlines.

<u>The work is organized as follows :</u>

I - Implementation and measurement of task τ_1 (large-number multiplication) to determine its WCET

II - Schedulability analysis of the complete task set

III - An exhaustive search for optimal non-preemptive schedules minimizing total waiting time.

2. Task τ_1 — Multiplication & Measurement

2.1 Implementation

Task τ_1 calculates product two large randomly generated numbers. The implementation uses a naive big-integer multiplication algorithm operating on digit arrays in base 10. Each operand consists of 1000 randomly generated digits.

The C program (multiplication.c) performs the following steps:

- Allocate two arrays of NUM_DIGITS (1000) integers for operands A and B
- Fill each digit position with a random digit [0, 9]
- Start high-resolution timer (QueryPerformanceCounter on Windows)
- Execute naive $O(n^2)$ digit-by-digit multiplication with carry propagation
- Stop timer and print elapsed time in microseconds to stdout

The program is compiled with gcc -O0 (no optimization) to ensure consistent and representative worst-case execution times.

2.2 Measurement Results

The measurement script (measurement.py) executes the compiled multiplication program 100 times and collects execution times. Results:

Statistic	Value (µs)
Minimum	2106.50
Q1 (25th percentile)	2124.40
Q2 (Median)	2196.35
Q3 (75th percentile)	2227.05
Maximum / WCET	2564.30
Mean	2194.18
Std. Deviation	77.23

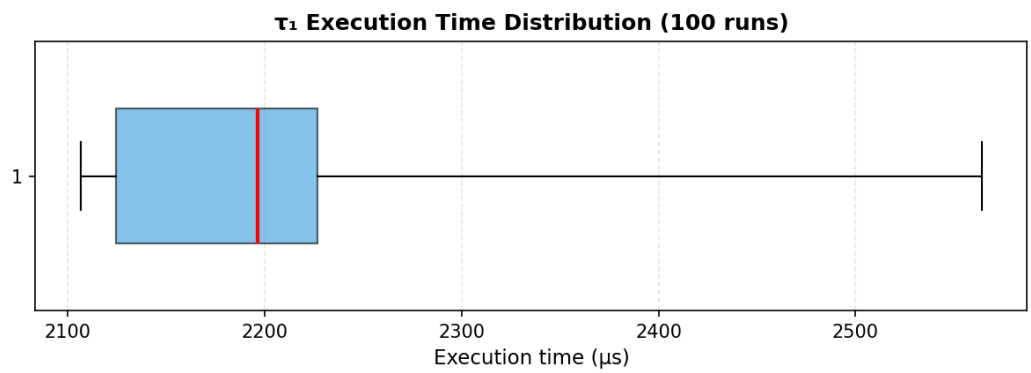

Figure 1: Box plot of τ_1 execution times over 100 runs.

Measurements were performed on a machine equipped with an AMD Ryzen 7 5800X processor (8 cores, 16 threads, 4.7 GHz boost frequency, Zen 3 architecture) with 32 GB of DDR4-3200 RAM, running Windows 11, under light load conditions (no resource-intensive applications active, machine connected to mains power). Since the effective clock frequency and the state of the processor's L1/L2/L3 caches are critical factors for execution duration, the measured WCET is intrinsically tied to this target hardware — a classical point in real-time systems specification.

2.3 Determination of C_1

The measured WCET (Worst-Case Execution Time) is 2564.30 µs = 2.56 ms. Taking the ceiling in milliseconds: $C_1 = \lceil 2.56 \rceil = 3$ ms.

This value is used as the computation time C_1 for task τ_1 in the scheduling analysis that follows. Using the ceiling ensures the scheduled time budget always covers the worst case.

3. Task Set & Schedulability Analysis

3.1 Task Set Definition

The system consists of 7 independent periodic tasks with implicit deadlines ($D_i = T_i$). The tasks are non-preemptive: once a task starts executing, it runs to completion without interruption.

Task	C_i (ms)	$T_i = D_i$ (ms)	$U_i = C_i/T_i$
τ_1	3	10	0.3000
τ_2	3	10	0.3000
τ_3	2	20	0.1000
τ_4	2	20	0.1000
τ_5	2	40	0.0500
τ_6	2	40	0.0500
τ_7	3	80	0.0375

3.2 Utilization Analysis

The total processor utilization is:

$$U = 3/10 + 3/10 + 2/20 + 2/20 + 2/40 + 2/40 + 3/80 = 0.9375$$

Since $U = 0.9375 \leq 1.0$, the necessary utilization condition for schedulability is satisfied.

The hyperperiod is $H = LCM(10, 10, 20, 20, 40, 40, 80) = 80$ ms. Within one hyperperiod, the total required execution time is 75 ms out of 80 ms available, leaving 5 ms of idle time.

3.3 Schedulability Proof

The utilization condition $U \leq 1$ is necessary but **not sufficient** for non-preemptive scheduling (a task set with $U \leq 1$ can still miss deadlines due to non-preemptive blocking effects). A constructive sufficient proof is therefore needed: demonstrating that an actual valid schedule exists over one hyperperiod. Since all tasks are periodic and independent, a schedule that is feasible over one hyperperiod repeats infinitely.

Our exhaustive search (Section 4) found a valid non-preemptive schedule with zero missed deadlines over the hyperperiod [0, 80] ms. This constructively proves that the task set is schedulable under non-preemptive scheduling.

Additionally, every job's response time $R_j = finish_j - release_j$ is verified to be $\leq$ its deadline D_j (see Section 5).

4. Non-Preemptive Schedule Search

4.1 Algorithm Description

The scheduling algorithm operates at the job level within one hyperperiod. The permutations are generated using Python's itertools.permutations(). It uses the following approach:

1. Generate all job instances for each task within the hyperperiod (29 jobs total).
2. Use itertools.permutations() to enumerate all n! orderings of the 7 task indices (defining a static priority ordering). For each permutation, run a greedy non-preemptive dispatcher.
3. The dispatcher, at each scheduling point, selects the highest-priority ready job (released and not yet scheduled) and runs it to completion.
4. If a job finishes after its deadline and the task is not in the 'allowed miss' set, the permutation is rejected as infeasible.
5. Among all feasible permutations, the one yielding the minimum total waiting time (sum of start − release over all jobs) is selected as optimal.

4.2 Computational Complexity

Let n be the number of tasks and J the total number of jobs in one hyperperiod.

- Job generation: $O(J)$ where $J = \Sigma(H/T_i)$ for all tasks.

- Number of priority permutations explored: n! (here 7! = 5040).
- For each permutation, the greedy scheduler runs in $O(J^2)$ in the worst case (for each of J scheduling points, we scan up to J remaining jobs to find the highest-priority ready one).
- Total complexity: $O(n! \times J^2)$. With n = 7 and J = 29, this gives 5040 × 841 = ~4.2 million operations, which completes in under a second on modern hardware.

This brute-force approach is feasible because n is small (7 tasks). For larger task sets, heuristic or branch-and-bound methods would be required.

4.3 Part 1 — No Missed Deadlines

The optimal non-preemptive schedule with no missed deadlines has a total waiting time of 168 ms and 5 ms of idle time. The best priority ordering found is: $\tau_3 > \tau_4 > \tau_1 > \tau_2 > \tau_5 > \tau_6 > \tau_7$.

Job	C	Release	Start	Finish	Deadline	Status
τ_3	2	0	0	2	20	OK
τ_4	2	0	2	4	20	OK
τ_1	3	0	4	7	10	OK
τ_2	3	0	7	10	10	OK
τ_1	3	10	10	13	20	OK
τ_2	3	10	13	16	20	OK
τ_5	2	0	16	18	40	OK
τ_6	2	0	18	20	40	OK
τ_3	2	20	20	22	40	OK
τ_4	2	20	22	24	40	OK
τ_1	3	20	24	27	30	OK
τ_2	3	20	27	30	30	OK
τ_1	3	30	30	33	40	OK
τ_2	3	30	33	36	40	OK
τ_7	3	0	36	39	80	OK
τ_3	2	40	40	42	60	OK
τ_4	2	40	42	44	60	OK
τ_1	3	40	44	47	50	OK
τ_2	3	40	47	50	50	OK
τ_1	3	50	50	53	60	OK
τ_2	3	50	53	56	60	OK
τ_5	2	40	56	58	80	OK
τ_6	2	40	58	60	80	OK
τ_3	2	60	60	62	80	OK
τ_4	2	60	62	64	80	OK
τ_1	3	60	64	67	70	OK
τ_2	3	60	67	70	70	OK
τ_1	3	70	70	73	80	OK
τ_2	3	70	73	76	80	OK

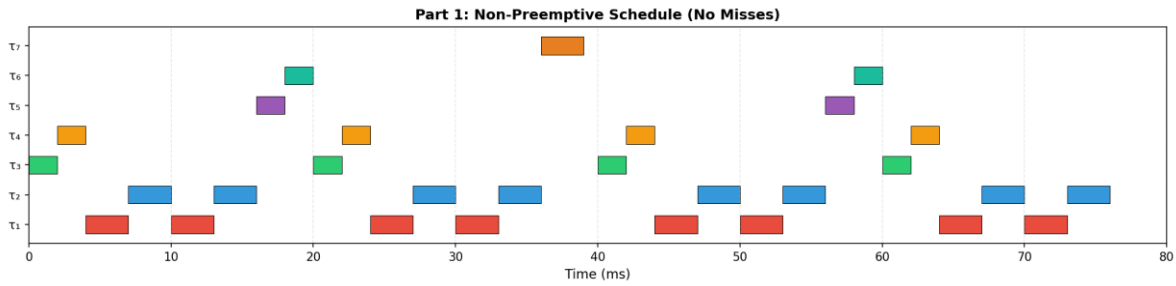

Figure 2: Gantt chart — Part 1 optimal schedule.

Relation between waiting time and idle time :

The assignment asks us to verify that minimizing the total waiting time W also maximizes the total idle time I. In fact, for a given task set analyzed over one hyperperiod, the *total execution demand* E is invariant: E = Σ C_i × (H / T_i) = 3×8 + 3×8 + 2×4 + 2×4 + 2×2 + 2×2 + 3×1 = 75 ms. Since every time unit in [0, H] is either execution or idle, we have the identity I = H − E = 80 − 75 = 5 ms for any valid schedule. The idle time is therefore a structural constant of the task set, not a function of the scheduling order.

What changes with the priority order is not the total idle time but its *distribution* over the hyperperiod, and therefore the waiting time W = Σ (start_j − release_j). An optimal ordering concentrates the idle slack into short contiguous intervals late in the hyperperiod rather than spreading it where it would push later jobs close to their deadlines. In our schedule, the 5 ms of idle appears during the interval [76, 80], after all jobs have completed — which is exactly the configuration that minimizes waiting. The verification idle + exec = 5 + 75 = 80 = H therefore confirms time-accounting correctness but does not discriminate between schedules; the true quality metric remains W = 168 ms, which our exhaustive search has shown to be minimal.

4.4 Part 2 — τ_5 May Miss a Deadline

When τ_5 is allowed to miss deadlines, the exhaustive search over all 5040 permutations finds that the optimal schedule is identical to Part 1, with total waiting time = 168 ms and zero missed deadlines.

Furthermore, a secondary search was performed to find the best schedule where τ_5 actually misses at least one deadline. No such permutation exists: with U = 0.9375 and only 5 ms of idle time per hyperperiod, the greedy dispatcher always manages to schedule τ_5 before its deadline regardless of priority ordering.

4.4.1 Why Part 2 = Part 1

This result is mathematically expected for two reasons:

- Relaxing a constraint (allowing τ_5 to miss) can only keep or improve the optimal value, never worsen it. Since Part 1 already found a feasible schedule, Part 2's search space is a superset and the same optimum is found.
- The system's structure — τ_5 has T_5 = 40 ms with only C_5 = 2 ms, producing just 2 jobs per hyperperiod — means τ_5 is never the bottleneck. The bottleneck tasks are τ_1 and τ_2 with the shortest periods (T = 10 ms) and highest utilization (0.30 each).

For the system to force a τ_5 miss, the total execution demand in any window of 40 ms would need to exceed 40 ms. The maximum demand in any 40 ms window is $4\times C_1 + 4\times C_2 + 2\times C_3 + 2\times C_4 + C_5 + C_6 + C_7 = 12 + 12 + 4 + 4 + 2 + 2 + 3 = 39$ ms < 40 ms, confirming that τ_5 can always be accommodated.

When would allowing τ5 to miss actually help ?

The result above is specific to the current task set, where the utilization U = 0.9375 leaves 5 ms of slack per hyperperiod and where the maximum execution demand within any 40 ms window is 39 ms < 40 ms. In a more heavily loaded system — for instance, if C_7 were increased from 3 to 7 ms (keeping T_7 = 80 ms, so U = 0.9875 remains schedulable in terms of total utilization) — the demand in a 40 ms window would rise to 43 ms > 40 ms. In that case the non-preemptive scheduler would be unable to meet all deadlines simultaneously: Part 1 would return no feasible schedule. Only Part 2, by allowing τ5 to miss, would keep the system operational.

Intuitively, the same effect would appear if τ_1 were measured on a slower target platform and its C_1 had to be raised accordingly: as C_1 grows, the slack shrinks, and at some point τ_5 can no longer fit within every 40 ms window. The "τ_5 may miss" option is therefore a graceful-degradation mechanism: it has no effect as long as the system is comfortably schedulable (as in our case), but it preserves feasibility when the workload is pushed to the edge of CPU capacity.

5. Job-Level Response Time Verification

For each job in the schedule, the response time R_j = finish_j − release_j must satisfy $R_j \le D_j$ (relative deadline = period T_i). The following table verifies this for all 29 jobs:

Job	Release	Finish	R_j	D_j	$R_j \le D_j$?
τ_3	0	2	2	20	OK
τ_4	0	4	4	20	OK
τ_1	0	7	7	10	OK
τ_2	0	10	10	10	OK
τ_1	10	13	3	10	OK
τ_2	10	16	6	10	OK
τ_5	0	18	18	40	OK
τ_6	0	20	20	40	OK
τ_3	20	22	2	20	OK
τ_4	20	24	4	20	OK
τ_1	20	27	7	10	OK

τ_2	20	30	10	10	OK
τ_1	30	33	3	10	OK
τ_2	30	36	6	10	OK
τ_7	0	39	39	80	OK
τ_3	40	42	2	20	OK
τ_4	40	44	4	20	OK
τ_1	40	47	7	10	OK
τ_2	40	50	10	10	OK
τ_1	50	53	3	10	OK
τ_2	50	56	6	10	OK
τ_5	40	58	18	40	OK
τ_6	40	60	20	40	OK
τ_3	60	62	2	20	OK
τ_4	60	64	4	20	OK
τ_1	60	67	7	10	OK
τ_2	60	70	10	10	OK
τ_1	70	73	3	10	OK
τ_2	70	76	6	10	OK

All 29 jobs satisfy their deadline constraints. The maximum response time observed is that of τ_7 (response = 39 ms, deadline = 80 ms).

6. Assumptions & Design Decisions

Non-preemptive execution : Once a task starts executing, it cannot be interrupted. This simplifies the scheduler but may increase waiting times.

Implicit deadlines : Each task's deadline equals its period ($D_i = T_i$).

Independent tasks : Tasks do not share resources or have precedence constraints.

WCET as C_1 : The measured WCET (2564.30 µs) is used as the computation time for τ_1. The ceiling function ($\lceil 2.56 \rceil$ = 3 ms) provides a safe upper bound.

No optimization flags : The C program is compiled with -O0 to avoid compiler optimizations that could reduce execution variability.

Exhaustive search over permutations : All 7! = 5040 task priority orderings are tested. This is feasible for 7 tasks and guarantees global optimality.

Hyperperiod analysis : The schedule is analyzed over one hyperperiod (80 ms = LCM of all periods). Since tasks are periodic and independent, a valid hyperperiod schedule repeats indefinitely.

Time unit convention : All scheduling parameters (C, T, D) are expressed in milliseconds. The measured WCET in microseconds is converted to milliseconds with ceiling rounding.

7. Conclusion

This report demonstrated a complete real-time scheduling solution for a 7-task periodic system. Key findings:

- Task τ_1 (1000-digit multiplication) has a measured WCET of 2564.30 µs, yielding C_1 = 3 ms.
- The total utilization U = 0.9375 satisfies the necessary schedulability condition ($U \leq 1$).
- An exhaustive search over all 5040 priority permutations found an optimal non-preemptive schedule with total waiting time = 168 ms and 5 ms idle time.
- All 29 jobs meet their deadlines in the optimal schedule.
- Allowing τ_5 to miss deadlines does not improve the schedule. No permutation of task priorities can even force τ_5 to miss, because the maximum execution demand in any 40 ms window (39 ms) is strictly less than 40 ms.